

Robust Guidance Law Design of Launch Vehicles via Deep Reinforcement Learning Algorithm

Chenhui Jia¹, Xiaodong Liu¹, Xiaomin Li¹, Qinghai Gong¹, Zhaolei Wang¹,
Ran Li¹

¹ National Key Laboratory of Aerospace Intelligent Control Technology, Beijing Aerospace
Automatic Control Institute, Beijing, China
championjch@sina.com

Abstract. The design of guidance laws for launch vehicles in the gale area necessitates a delicate balance between achieving high control accuracy, ensuring robustness against significant disturbances, and minimizing overload to safeguard against potential damage to the launch vehicle's structure. In this paper, we develop network formed adaptive guidance laws for various flight tasks, which are trained with a reinforcement learning algorithm to address the challenges mention above. The simulation results demonstrate that the guidance laws trained with our algorithm exhibit robust characteristics under the influence of various wind disturbances and accurately track the desired flight trajectory with high precision throughout the flight process. Furthermore, in the gale area, the guidance laws enable the launch vehicle to operate with reduced overload while maintaining an acceptable level of tracking accuracy. This algorithm represents a promising candidate for designing guidance laws for launch vehicles in the gale area or under unknown disturbances during the flight process.

Keywords: Adaptive Guidance Law, Reinforcement Learning, Soft Actor-Critic, Overload Reduction, Launch Vehicles.

1 Introduction

Launch vehicle flight control design, as a pivotal technology in the control systems of launch vehicles, has garnered extensive attention across the aerospace field [1-3]. During the flight control process of the ascending phase after the launch of the launch vehicle, the engineers not only need to consider the influence of wind interference but also need to consider the importance of reducing the overload of the launch vehicle in the gale area. Due to the structural characteristics of the launch vehicle with large slenderness ratio, it is necessary to reduce the overload of the launch vehicle to a safe range to prevent the launch vehicle from the damage of the vehicle body. As the overload reduction may lead to the loss of the orbit injection accuracy during the flight process, especially under the disturbance of the wind in the gale area, we may consider the guidance of the launch vehicle as a trade-off between the control accuracy and the overload reduction. Therefore an adaptive guidance law is required which not can only control the launch vehicle under the condition of unknown high disturbance, but

also balance the control accuracy and the overload reduction to make the flight process of the launch vehicle achieves a satisfied flight process.

As a new intelligent planning and decision algorithm, deep reinforcement learning algorithm (DRL algorithm for short) which combines the traditional reinforcement learning decision-making algorithm and the deep learning algorithm, has greatly enhanced the decision-making ability of agents through deep neural networks. In recent years, deep reinforcement learning has achieved great achievements on chess games [4], complex RTS computer games [5], multi-agent games [6], robots control [7], and autopilots [8].

Among variety of the DRL algorithms, Soft Actor-Critic algorithm [9-10] as an off-policy stochastic policy gradient algorithm [11] which is utilized to solve the problems of continuous action space problems. Compared with the deterministic policy gradient algorithm, such as DDPG [12] or TD3 [13], SAC algorithm demonstrates its more powerful exploration ability which encourages the agent to explorer new policies while keeping the rewards not significantly reduced. Besides, SAC algorithm inherits the experience replay mechanism of the off-policy DRL algorithm and the clipped double Q learning mechanism of TD3 algorithm so that the algorithm will realize the fast convergence during the training process without the overestimation of the accumulate returns. Due to the characteristics above, SAC algorithm is considered as a suitable method to solve the problems of continuous action space with complex reward. In this paper, we train the guidance laws of launch vehicle with SAC algorithm to solve the desired trajectory tracking problem in different disturbances of wind field and the overload relief problem during the flight process after launch.

The rest of this paper is organized as follows. In Section 2, we will build the mathematic model of a launch vehicle and construct Markov decision process of two flight task. In Section 3, we will introduce the design of guidance law training agent basing on SAC algorithm. In Section 4, we will carry out simulations of the two flight tasks with the guidance laws designed with our algorithm. We will draw a conclusion in Section 5.

2 Problem Statement

In this paper, we consider the Ares-I launch vehicle [14] as the object to realize the position tracking control with load relief strategy under different conditions of wind fields. Without losing generality, we only consider the flight control of longitudinal plane. During the ascending flight phase, the model of Ares-I launch vehicle in the 2-D longitudinal plane is presented as:

$$\begin{cases} \begin{bmatrix} \dot{x} \\ \dot{z} \end{bmatrix} = \begin{bmatrix} v_x \\ v_z \end{bmatrix} \\ m \begin{bmatrix} \dot{v}_x \\ \dot{v}_z \end{bmatrix} = T_{b2g}(\mathbf{F}_{aero} + \mathbf{F}_{Thrust}) - m\mathbf{g} \end{cases} \quad (1)$$

where m is the current mass of the launch vehicle, $x > 0$ represents the current altitude of the launch vehicle, $z < 0$ is the current geographic distance between the

launch vehicle and the launch point. v_x and v_z denote the velocity components of the launch vehicle on the x-axis and z-axis, respectively. $\mathbf{F}_{aero}$ represents the aerodynamic force vector of the launch vehicle during the flight process. $\mathbf{F}_{thrust} = [F_{th}, 0]^T$ represents the current thrust vector of the launch vehicle. $\mathbf{g} = [g, 0]^T$ is the gravitational acceleration vector, where g is the gravitational acceleration of the current position. In this paper, ignoring the influence of earth curvature, g is calculated as:

$$g = g_0 \frac{R^2}{(x+R)^2} \quad (2)$$

where R is the average earth radius, g_0 is the gravitational acceleration at the sea level. The matrix $\mathbf{T}_{b2g}$ represents the convert matrix from the inertial coordinate to the body coordinate of the launch vehicle, where:

$$\mathbf{T}_{b2g} = \begin{bmatrix} \cos\varphi & \sin\varphi \\ -\sin\varphi & \cos\varphi \end{bmatrix} \quad (3)$$

where φ is the pitch angle of the launch vehicle. In this paper, we assume that the attitude control loop of the launch vehicle is ideal and consider φ as the output of the guidance law to control the launch vehicle follow the desired trajectory.

The aerodynamic force vector $\mathbf{F}_{aero}$ is calculated as:

$$\mathbf{F}_{aero} = \mathbf{T}_{b2g} [F_{aero_basic} - C_A q S, -C_N q S \alpha]^T \quad (4)$$

where F_{aero_basic} is the basic axial aerodynamic force which is a nonlinear function of altitude. S is the reference area of the launch vehicle, $q = 0.5\rho(v_x^2 + v_z^2)$ is the dynamic pressure of the launch vehicle, ρ is the atmospheric density of the current altitude calculated according to the U.S. Standard Atmosphere model, 1976 (USSA, 1976) [16], α is the attack angle.

In this paper, we only consider the effect of horizontal wind on the control of launch vehicle. 32 types of wind field are considered in this simulation. The curves of wind speed at different altitude of all wind fields are shown in Fig.1. It is shown that the wind speed at the altitude of 10.0km-15.0km is relatively high in the working range of the launch vehicle. When designing the guidance law, we consider this area as a gale area in which we will control the launch vehicle with a lower overload.

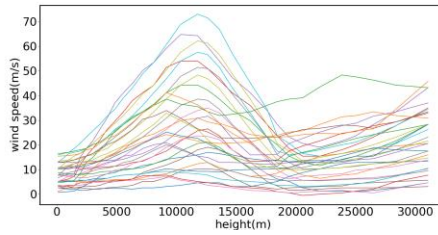


Fig.1. 32 types of wind field in this paper.

C_A and C_N in Eq.(4) are the axial aerodynamic force coefficient and normal aerodynamic force coefficient of the launch vehicle, respectively. In this paper, C_A and C_N are considered as the nonlinear function of Mach number at the current altitude. In this paper, C_A and C_N are calculated according to the appendix of [15].

Consider the process of trajectory tracking control as a Markov Decision Process (MDP) [11], we assume that the position and velocity of the launch vehicle in the next control period is only determined by the current the position and velocity of the launch vehicle and the control signal of the guidance loop. In this paper, as is mentioned above, the pitch angle is considered as the output signal of the guidance law.

Assume that the launch vehicle tracks the desired trajectory with zero tracking error, the current position and velocity of the launch vehicle are presented as $[x_d, z_d]^T$ and $[v_{xd}, v_{zd}]^T$, respectively. In this paper, the goal of our guidance law design is to make the launch vehicle track the desired trajectory with high tracking accuracy and low normal overload. To make the state variable directly represent the effect of position control accuracy and the normal acceleration N_z (represents the acceleration of the launch vehicle at z-axis), we utilize the position tracking error and the velocity tracking error to combine the state vector $\mathbf{s}_t$, described as:

$$\mathbf{s}_t = [k_1 e_x, k_2 e_z, k_3 e_{vx}, k_4 e_{vz}, k_5 \varphi, k_6 N_z]^T \quad (6)$$

where

$$\begin{cases} e_x = x_d - x \\ e_z = z_d - z \\ e_{vx} = v_{xd} - v_x \\ e_{vz} = v_{zd} - v_z \end{cases} \quad (7)$$

$\mathbf{K} = [k_1, k_2, k_3, k_4, k_5, k_6]^T$ is the state weight vector to adjust each variable of the state vector to within or near the range of [-1,1].

The action vector $\mathbf{a}_t$ of the Markov Decision Process is set as:

$$\mathbf{a}_t = [\Delta\varphi]^T \quad (8)$$

where $\Delta\varphi$ is the increment of the pitch angle between the current control step and the previous control step. In order to compare the performance difference between the guidance law with load relief and the guidance law without load relief, in this paper we design two different tasks and train different neural network controllers through SAC reinforcement learning algorithm:

Task 1: The launch vehicle tracks the desired trajectory with the minimum tracking error without considering the normal overload. The flight time costs 90s. The reward function of the task is set as:

$$r_t(\mathbf{s}_t, \mathbf{a}_t) = -\mathbf{e}_t^T \mathbf{P}_1 \mathbf{e}_t \quad (9)$$

where $\mathbf{e}_t = [e_x, e_z, e_{vx}, e_{vz}]^T$, $\mathbf{P}_1$ is a positive definite matrix. When designing the guidance law for Task 2, the effect of the guidance law in Task 1 will be considered as a reference to compare the effect of the guidance law in Task 2 in terms of load reduction effect and control accuracy.

Task 2: The launch vehicle tracks the desired trajectory with the balance of minimal tracking error and minimal normal overload. Considering that high normal overload mainly occurs in the gale area. When designing the reward function, we add a penalty term related to normal overload to the reward function. The reward function is set as:

$$r_t = \begin{cases} -\mathbf{e}_t^T \mathbf{P}_1 \mathbf{e}_t - 0.01\alpha \cdot q & 10^4 < x < 1.5 \times 10^4 \\ -\mathbf{e}_t^T \mathbf{P}_1 \mathbf{e}_t & \text{otherwise} \end{cases} \quad (10)$$

3 The Design of Guidance Law Training Algorithm

In this paper, we utilized SAC algorithm to train the guidance law of under the condition of Task 1 and Task 2. The structure of SAC agent designed in our paper is shown in Fig.2. As a stochastic policy gradient algorithm, the actor network is utilized to generate an optimum policy $\pi \sim N(\boldsymbol{\mu}_t, \boldsymbol{\sigma}_t)$ to acquire the optimum from the environment, where $\boldsymbol{\mu}_t$ and $\boldsymbol{\sigma}_t$ are the output vectors of the actor network. The action vector $\mathbf{a}_t$ is sampled from the policy conforms to a normal distribution with an expectation vector $\boldsymbol{\mu}_t$ and standard variance vector $\boldsymbol{\sigma}_t$. value networks are utilized to evaluate the expectation of accumulated return $V(\mathbf{s}_t)$ under the state $\mathbf{s}_t$. The outputs of Q networks are utilized to evaluate the accumulated return the agent gets with the action $\mathbf{a}_t$ under the state $\mathbf{s}_t$. Similar as TD3 algorithm, the structure of twin Q networks in SAC algorithm is to prevent the overestimation of the accumulated value.

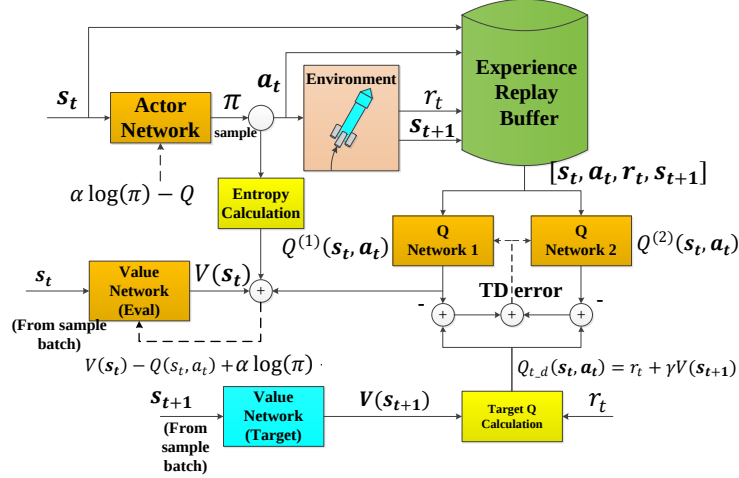


Fig.2. The guidance law training system basing on SAC algorithm.

Assume the accumulated return from the time t is $Q_{t,d}(\mathbf{s}_t, \mathbf{a}_t)$, According to Bellman equation, $Q_{t,d}(\mathbf{s}_t, \mathbf{a}_t)$ is written as:

$$Q_{t,d}(\mathbf{s}_t, \mathbf{a}_t) = r_t(\mathbf{s}_t, \mathbf{a}_t) + \gamma \mathbb{E}_{\mathbf{s}_{t+1} \sim p} (V(\mathbf{s}_{t+1})) \quad (11)$$

In SAC algorithm, the agent aims to acquire not only the maximum reward but also the maximum entropy. Taking the policy entropy as consideration, the desired value of $V_d(\mathbf{s}_t)$ is described as:

$$V_d(\mathbf{s}_t) = \mathbb{E}_{a_t \sim \pi} (Q_t(\mathbf{s}_t, \mathbf{a}_t) - \alpha \log \pi(\mathbf{a}_t | \mathbf{s}_t)) \quad (12)$$

where α is the temperature coefficient to measure the importance of policy entropy. $Q_t(\mathbf{s}_t, \mathbf{a}_t)$ in Eq. (14) is calculated as:

$$Q_t(\mathbf{s}_t, \mathbf{a}_t) = \underset{i=1,2}{\operatorname{argmin}} \{Q_t^{(i)}(\mathbf{s}_t, \mathbf{a}_t)\} \quad (13)$$

The loss function of value network (eval) is the square error of $V_d(\mathbf{s}_t)$ and $V(\mathbf{s}_t)$, described as:

$$loss_V(t) = 0.5 (V_d(\mathbf{s}_t) - V(\mathbf{s}_t))^2 \quad (14)$$

According to Bellman equation, the loss function of each Q network is set as:

$$loss_{Q^{(i)}}(t) = 0.5 (Q_{t-d}(\mathbf{s}_t, \mathbf{a}_t) - Q_t^{(i)}(\mathbf{s}_t, \mathbf{a}_t))^2 \quad (15)$$

Consider the network generates an action $\mathbf{a}_t$ with a normal distributed noise with standard variance σ_t , the loss function of the actor network is written as:

$$loss_\pi(t) = \mathbb{E}_{\mathbf{s}_t \sim p} (\alpha_0 \log(p_\pi(\mathbf{a}_t | \mathbf{s}_t)) - Q_t(\mathbf{s}_t, \mathbf{a}_t)) \quad (16)$$

where $p_\pi(\mathbf{a}_t | \mathbf{s}_t)$ denotes the probability density of action $\mathbf{a}_t$ under the state $\mathbf{s}_t$ with the distribution $\pi \sim N(\boldsymbol{\mu}_t, \boldsymbol{\sigma}_t)$. The loss function shows that compared with actions which acquires high returns with high probability density (e.g. $\mathbf{a}_t \approx \boldsymbol{\mu}_t$), the agent prefers to the actions which acquire similar accumulated returns with a lower probability density. The mechanism makes the algorithm more inclined to explore new policies while keeping the accumulated returns not reduced.

In Ref. [10], the author modified the algorithm with an adaptive temperature coefficient α_0 . The loss function of α is set as:

$$loss_\alpha(t) = \mathbb{E}_{a_t \sim \pi} (-\alpha_0 \log(p_\pi(\mathbf{a}_t | \mathbf{s}_t)) - \alpha_0 \mathcal{H}) \quad (17)$$

where $\mathcal{H}$ is the dimension of the action vector.

In this paper, all networks are set as fully-connected multi-layer networks. The output of each layer is written as:

$$\mathbf{y} = \mathbf{h}(\mathbf{w}^T \mathbf{x} + \mathbf{b}) \quad (18)$$

where $\mathbf{x}$ and $\mathbf{y}$ are the input vector and the output vector of the layer, respectively. $\mathbf{w}$ and $\mathbf{b}$ are the parameter matrixes (or vectors) of the layer. $\mathbf{h}$ is the activate function. The parameters of the target networks are updated with the update law below:

$$\mathbf{w}_{tar,n}^{(t+1)} = (1 - \tau) \mathbf{w}_{tar,n}^{(t)} + \tau \mathbf{w}_{eval,n}^{(t)} \quad (19)$$

where $\mathbf{w}_{eval,n}^{(t)}$ and $\mathbf{w}_{tar,n}^{(t)}$ are the parameters of the n th eval network and the n th target network at time t , respectively. After all networks are converged, the actor network can be utilized to generate actions as the guidance law.

The training process of the algorithm is shown in Table 1.

Table 1. The training process of SAC algorithm

Algorithm 1: Load relief guidance law training with SAC algorithm

```

Initialize all parameters of networks in Fig.2 with random values and initial the state of the launch vehicle.
while ( $steps < batch\_size$ ) do
    Generate actions with the actor network and run Markov decision process in the environment, zip all information as the form  $[\mathbf{s}_t, \mathbf{a}_t, r_t, \mathbf{s}_{t+1}]$ .
end while
while ( $episode < episode_{max}$ ) do
    Initial the state of the launch vehicle with initial state  $\mathbf{s}_0$ .
    for  $t = 1, T$  do
        Calculate  $\boldsymbol{\mu}_t$  and  $\boldsymbol{\sigma}_t$  with actor network with input state  $\mathbf{s}_t$ , and generate action  $\mathbf{a}_t \sim N(\boldsymbol{\mu}_t, \boldsymbol{\sigma}_t)$ .
        Run Markov decision process in the environment, zip all information as the form  $[\mathbf{s}_t, \mathbf{a}_t, r_t, \mathbf{s}_{t+1}]$ .
        if  $t \% update\_frequency == 0$ , do
            Calculate  $V(\mathbf{s}_t)$  and  $V(\mathbf{s}_{t+1})$  with eval value network and target value network, respectively.
            Calculate  $Q_t^{(i)}(\mathbf{s}_t, \mathbf{a}_t)$  with Q networks, Calculate  $Q_{t,d}(\mathbf{s}_t, \mathbf{a}_t)$  according to Eq.(11).
            Update parameters of all networks with Adam optimizer and loss functions in Eq.(14)~(17).
            Update parameters of target value network according to Eq.(19).
        end if
    end for
end while

```

4 Simulation Result

During the simulation, we utilize Runge-Kutta method to solve the differential equations in Eq.(1). The control period is set as 0.1s. For each task, 4 types of wind field are set in the training environment as a priori knowledge of our training process, while the other 28 types of wind field are set in the test environment to test the generalization of our algorithm. Parameters of the launch vehicle are set in Ref. [15].

The actor network is set as the controller during the flight process after the networks are converged. Assume the coordinate of the launch point is at (0km, 0km). According to the standard flight trajectory, the launch vehicle will reach the point nearly (22.014km, -15.308km) after 90s of flight. During the training process, all hyper-parameters of our algorithm for Task 1 and Task 2 are set in Table 2.

The training processes of the guidance law for the two tasks are shown in Fig.3. The blue curves indicate the total reward of the current episode and the red curves indicate average total reward of the last 100 episodes. After about 1000 episodes of training, all networks and the temperature coefficient are converged and the reward of

the episode gets nearly zero, which means the control error of the position and velocity are converged with the guidance law designed with our algorithm.

Table 2. Hyper-parameters of SAC algorithm

Parameters	Definition	Value
$N^{(\pi)}$	Number of units of actor network in each hidden layer	[256, 256]
$N^{(V)}$	Number of units of actor network in each hidden layer	[256, 256]
$N^{(Q)}$	Number of units of actor network in each hidden layer	[256, 256]
η_{net}	Learning rate of networks	0.0001
η_{α}	Learning rate of temperature coefficient	0.0001
γ	Discount factor	0.99
τ	Soft-replacement parameter of target value network	0.005

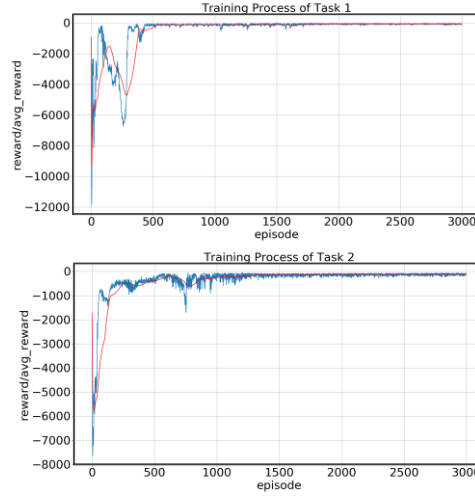


Fig. 3. The training processes of the agents in the two tasks above.

Fig.4 shows the effect of the trajectory tracking with our guidance law under different wind fields. The red curve represents the desired trajectory, and the curves of other colors shows the flight trajectory of the launch vehicle under different wind field. It is shown that the flight curves of the launch vehicle in different conditions are almost coincide with the desired trajectory. The maximum tracking errors in the direction of x-axis and z-axis are less than 100m. Consider the maximum speeds of the x-axis and the z-axis are larger than 600m/s and 400m/s, respectively, and the control step is set as 0.1s, the tracking error of the guidance law almost reaches its optimum state. Fig.5 shows the effect of the trajectory tracking with our guidance law under different wind fields. It is shown that in different conditions, the launch vehicle tracks the desired trajectory with a lower overload. The maximum tracking errors in the

direction of x-axis and z-axis are less than 200m, and the maximum overload of the launch vehicle in gale area is reduced by nearly 40%.

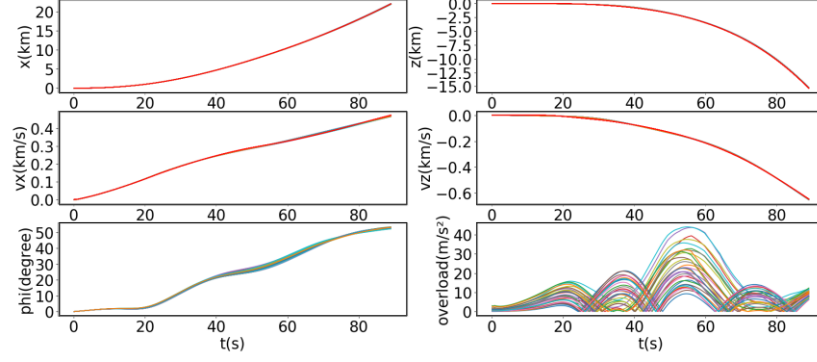


Fig.4. The control result of the guidance law in Task 1. The two upper figures show the position curve of the launch vehicle in x-axis and z-axis in different wind fields, respectively; the two middle figures in the middle indicate the velocity curve of the launch vehicle in x-axis and z-axis in different wind fields, respectively; the bottom left figure shows the pitch angle curve of the launch vehicle in different wind fields, and the bottom right figure shows the overload curve of the launch vehicle in different wind fields.

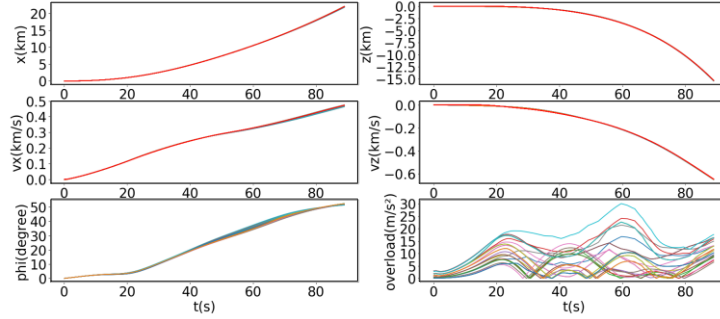


Fig.5. The control result of the guidance law in Task 2. The implication of each sub figure is the same as that of Fig.4.

5 Conclusion

In this paper, SAC reinforcement learning algorithm is utilized to train neural-networked guidance laws of for two launch vehicle flight tasks. According to the simulation result, with the effect of our guidance law, the launch vehicle tracks the desired trajectory with a satisfied tracking error during the flight process under different disturbance of wind fields in Task 1, moreover, the launch vehicle can still accu-

rately track the desired trajectory with a lower overload under the effect of our guidance law in Task 2. The algorithm presented in this paper is promising candidate for developing guidance laws for launch vehicles, effectively enabling these launch vehicles to adhere to a predetermined trajectory with reduced overloads.

This work is supported by the National Natural Science Foundation of China (Grant No. U21B2028).

References

1. R. M. Pinson, T. L. Schmitt, and J. M. Hanson. Development of a Smooth Trajectory Maneuver Method to Accommodate the Ares I Flight Control Constrains. *AIAA 2008-6292, AIAA Guidance, Navigation, and Control Conference, Honolulu, Hawaii, August 18-21, 2008*.
2. M. Whorton, C. Hall, and S. Cook. Ascent Flight Control and Structural Interaction for the Ares-I Crew Launch Vehicle. *AIAA 2007-1780*, April 2007.
3. R. L. James. A Three-Dimensional Trajectory Simulation Using Six Degrees of Freedom with Arbitrary Wind,” *NASA TN D-641*, 1961.
4. Silver, D., Huang, et al., Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587), 484.
5. O. Vinyals, T. Ewalds, S. Bartunov, et al. StarCraft II: A New Challenge for Reinforcement Learning. *arXiv preprint, arXiv:1708.04782*, 2017.
6. C. Berner, G. Brockman, B. Chan, et al. Dota 2 with Large Scale Deep Reinforcement Learning. *arXiv preprint, arxiv:1912.06680*, 2019.
7. K. Kamali, I. A. Bonev, C. Desrosiers. Real-time Motion Planning for Robotic Teleoperation Using Dynamic-goal Deep Reinforcement Learning. *17th Conference on Computer and Robot Vision (CRV)*, 2020.
8. A. Sallab, M. Abdou, E. Perot, S. Yogamanl. Deep Reinforcement Learning framework for Autonomous Driving. *arXiv preprint, arXiv:1704.02532v1*, 2017.
9. T. Haarnoja, A. Zhou, P. Abbeel, S. Levine. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. *arXiv preprint, arxiv:1801.01290v2*, 2018.
10. T. Haarnoja, A. Zhou, K. Hartikainen, et al. Soft Actor-Critic Algorithms and Applications. *arXiv preprint, arxiv:1812.05905v2*, 2019.
11. Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction, Second edition*. The MIT Press, 2017.
12. Lillicrap, Timothy P. Continuous control with deep reinforcement learning. *arXiv preprint, arXiv:1509.02971*, 2015.
13. S. Fujimoto, H. Van Hoof, and D. Meger. Addressing function approximation error in actor-critic methods. *ICML*, 2018.
14. Butt, A., Popp, C. G., Pitts, H. M. and Sharp, D. J., “NASA Ares I Launch Vehicle Roll and Reaction Control Systems Design Status,” *NASA Technical Reports Server*, August 2009.
15. W. Du. Dynamic modeling and ascent flight control of Ares-I Crew Launch Vehicle [D]. Iowa State University, 2010.
16. *U.S. Standard Atmosphere 1976*. National Oceanic and Atmospheric (NOAA), 1976.